



УНИВЕРЗИТЕТ У НИШУ
ЕЛЕКТРОНСКИ ФАКУЛТЕТ



ГЕНЕРИСАЊЕ ОПИСА СЛИКА

Технолошки пројекат
Студијски програм: Вештачка интелигенција и машинско учење
Модул:

Студенти:

Јована Стојановић, бр. инд. 1902
Лука Гавриловић, бр. инд. 1823

Ментор:

Проф. др Александар Милосављевић

Ниш, октобар 2025. година

САДРЖАЈ

1. УВОД.....	5
2. ТЕОРИЈСКЕ ОСНОВЕ И ПРЕГЛЕД ПОСТОЈЕЋИХ ПРИСТУПА.....	6
2.1. Аутоматско генерисање описа слика.....	6
2.2. Савремени модели и <i>Neural Image Caption</i> (NIC).....	6
3. АРХИТЕКТУРА.....	8
3.1. Енкодер–декодер.....	8
3.2. Енкодер VGG-16 неуронска мрежа.....	8
3.3. Декодер - LSTM рекурентна неуронска мрежа.....	10
3.4. Евалуација модела.....	13
4. <i>PROOF-OF-CONCEPT</i> ПРОЈЕКАТ.....	14
4.1. Коришћене технологије.....	14
4.2. Архитектура апликације.....	14
4.3. Екстракција карактеристика из слика (<i>feature extraction</i>).....	17
4.4. Обрада наслова (<i>Caption Processor</i>).....	18
4.5. Генератор података (<i>Data Generator</i>).....	18
4.6. Модел за генерисање наслова (<i>Caption Model</i>).....	19
4.7. Главни програм (<i>Main</i>).....	20
5. ПРЕГЛЕД РЕЗУЛТАТА.....	24
5.1. Анализа грешака.....	24
5.2. Поређење стратегија.....	26
6. ЗАКЉУЧАК.....	27
ЛИТЕРАТУРА.....	28

ГЕНЕРИСАЊЕ ОПИСА СЛИКЕ

САЖЕТАК

Развој вештачке интелигенције омогућио је интеграцију различитих когнитивних способности, међу којима се посебно издваја повезивање рачунарског вида и обраде природног језика. Један од најзначајнијих изазова у овој области је аутоматско генерисање описа слика (*Image Captioning*), чији је циљ да систем на основу улазне слике самостално креира смислен, граматички исправан и контекстуално релевантан опис. У овом раду представљен је *proof-of-concept* систем који комбинује конволуционе неуронске мреже (CNN) за екстракцију визуелних карактеристика и рекурентне неуронске мреже (RNN), односно LSTM архитектуру, за генерисање текстуалних описа.

Коришћени скупови података садрже парове слика и њихових описа, чиме се модел обучава да повеже визуелни садржај са језичком репрезентацијом. Рад примењује *encoder-decoder* архитектуру, технике *tokenization* и *embedding* за нумеричко представљање текста, као и BLEU метрику за евалуацију квалитета генерисаних описа.

Резултати указују да се комбиновањем метода дубоког учења може успешно изградити систем који спаја визуелно препознавање и језичко разумевање, што представља значајан корак ка развоју интелигентних мултимодалних апликација.

Кључне речи: вештачка интелигенција; обрада слика; обрада природног језика; конволуционе неуронске мреже; рекурентне неуронске мреже

IMAGE CAPTIONING

ABSTRACT

The development of artificial intelligence has enabled the integration of various cognitive abilities, among which the connection between computer vision and natural language processing stands out. One of the most significant challenges in this field is automatic image caption generation (*Image Captioning*), whose goal is to enable a system to independently create a meaningful, grammatically correct, and contextually relevant description based on an input image. This paper presents a *proof-of-concept* system that combines *Convolutional Neural Networks* (CNN) for visual feature extraction and *Recurrent Neural Networks* (RNN), specifically the LSTM architecture, for text generation.

The datasets used contain pairs of images and their corresponding descriptions, allowing the model to learn the relationship between visual content and linguistic representation. The paper employs an *encoder-decoder* architecture, *tokenization* and *embedding* techniques for numerical text representation, as well as the BLEU metric for evaluating the quality of generated descriptions.

The results indicate that by combining deep learning methods, it is possible to successfully build a system that integrates visual recognition and language understanding, representing an important step toward the development of intelligent multimodal applications.

Keywords: artificial intelligence; image processing; natural language processing; convolutional neural networks; recurrent neural networks

1. УВОД

Развој вештачке интелигенције довео је до интеграције различитих когнитивних способности, међу којима је посебно значајна веза између рачунарског вида и обраде природног језика (*Natural Language Processing* – NLP). Један од најактуелнијих задатака у овој области је аутоматско генерисање описа слика (*Image Captioning*), чији је циљ да систем на основу улазне слике самостално формира смислен, граматички исправан и контекстуално релевантан опис.

Проблематика овог истраживања лежи у сложености повезивања визуелних и језичких информација. Модел мора да идентификује објекте на слици, разуме њихове међусобне односе и преведе их у текст који има јасно значење. Оваква интеграција представља изазов јер захтева спајање техника дубоког учења из различитих области - обраде слике и обраде текста.

У овом раду представљен је proof-of-concept систем који комбинује конволуционе неуронске мреже (*Convolutional Neural Networks* - CNN) за екстракцију визуелних карактеристика и рекурентне неуронске мреже (*Recurrent Neural Networks* - RNN), конкретно LSTM (*Long Short-Term Memory*) архитектуру, за генерисање текстуалних описа. Материјал који је коришћен обухвата скупове података са паровима слика и њихових описа, који омогућавају учење везе између визуелног садржаја и језичке репрезентације.

Област истраживања обухвата примену техника дубоког учења у мултимодалним системима који повезују визуелни и језички домен.

Циљ рада је развој модела који ће омогућити разумевање слика и њихово језичко описивање, као и процена ефикасности примене савремених архитектура у том процесу.

У раду се примењују следеће методе:

- Конволуционе неуронске мреже (CNN)
за екстракцију визуелних карактеристика,
- Рекурентне неуронске мреже (RNN) и LSTM јединице
за генерисање текстуалних описа,
- *Encoder-Decoder* архитектура
за повезивање визуелног и језичког дела модела,
- *Tokenization* и *Embedding* технике
за нумеричко представљање текста и
- BLEU метрика
за евалуацију квалитета генерисаних описа.

Овим приступом рад показује како се комбиновањем различитих метода дубоког учења може постићи систем који спаја визуелно препознавање и језичко разумевање, што представља основу за будуће интелигентне мултимодалне апликације.

2. ТЕОРИЈСКЕ ОСНОВЕ И ПРЕГЛЕД ПОСТОЈЕЋИХ ПРИСТУПА

2.1. Аутоматско генерисање описа слика

Развој дубоког учења омогућио је значајан напредак у областима рачунарског вида (енгл. *Computer Vision*) и обраде природног језика (енгл. *Natural Language Processing*). Аутоматско генерисање описа слика (енгл. *Image Captioning*) представља значајан проблем вештачке интелигенције који повезује две области. Овај задатак подразумева да рачунар генерише смислен и граматички исправан опис слике која је дата на улазу.

Генерисање описа слике обухвата бројне практичне примене које могу имати позитиван утицај на свакодневни живот. Примена *Image Captioning*-а може помоћи особама са оштећењем вида да боље разумеју садржај слика на интернету [1], може омогућити аутоматизовано описивање и организацију слика у великим колекцијама, побољшану претрагу мултимедијалних садржаја и подршку у развоју интелигентних система који разумеју визуелне податке.

Овај задатак је доста сложенији од класичног проблема класификација слика или детекције објеката који су у главном фокусу рачунарског вида. Опис слике не треба само да уочи који објекти се налазе на њој, већ и да нагласи у каквом су односу објекти, као и да обухвати атрибуте и активности који су садржани на слици.

2.2. Савремени модели и *Neural Image Caption* (NIC)

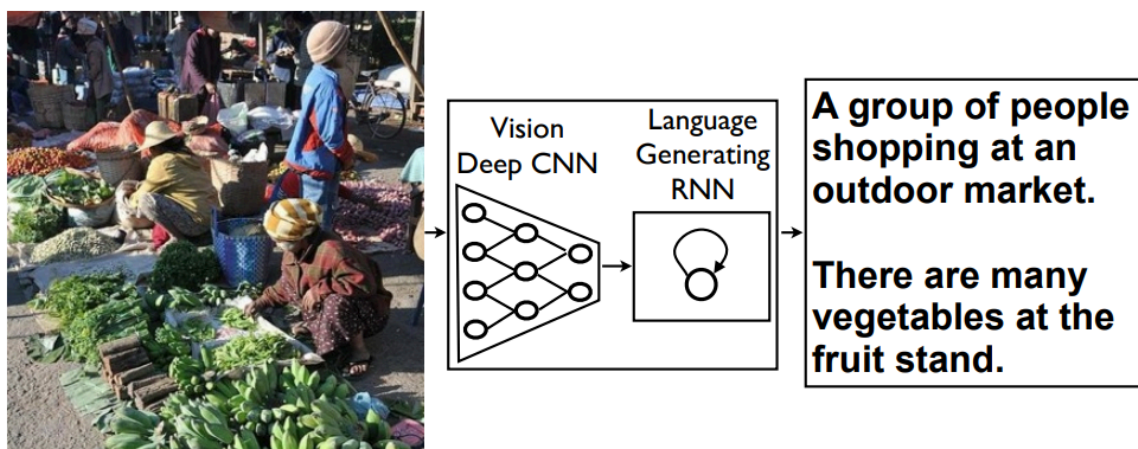
Традиционални приступи генерисању описа заснивали су се на одвојеним компонентама за детекцију објеката и ручно конструисаним правилима за формирање реченица. Међутим, овакав приступ био је ограничен и тешко прилагодљив различитим доменима. Са појавом конволуционих неуронских мрежа (CNN), које су постале стандард у анализи слика, и рекурентних неуронских мрежа (RNN), које се користе за моделовање секвенци, постало је могуће развити системе који директно уче како да повежу визуелне карактеристике слике са текстуалним описима.

У овом раду биће искоришћен модел који као улаз прима слику I и учи да максимизује вероватноћу $p(S|I)$ генерисања циљне секвенце речи $S = \{S_1, S_2, \dots\}$, где свака реч S_i потиче из унапред дефинисаног речника и адекватно описује дату слику.

Сличан проблем је постојао и у области машинског превођења. Други низ година превод је био заснован на низу одвојених корака - превођење појединачних речи, њихово поравнање и реорганизација редоследа. Међутим, савремени приступи су показали да се превод може реализовати много једноставније коришћењем рекурентних неуронских мрежа које су постизале изузетне резултате. У таквим системима, RNN „енкодер“ обрађује улазну реченицу и претвара је у опширну векторску репрезентацију фиксне дужине, која се затим користи као почетно скривено стање RNN „декодера“ задуженог за генерисање циљне реченице.

На овом принципу биће заснован и приступ у овом раду, са кључном разликом што улогу енкодера преузима дубока конволуциона неуронска мрежа. Истраживања показују да CNN може да извуче богату репрезентацију слике и представи је као

вектор фиксне дужине, који се даље може користити за различите задатке у области рачунарског вида [2]. Стога, CNN посматрамо као „енкодер слике“, који се прво обучава на задатку класификације слика, а затим се излаз последњег скривеног слоја користи као улаз у RNN декодер задужен за генерисање реченица. Овај модел називамо *Neural Image Caption* (NIC), приказан је на слици 1.



Слика 1. NIC модел који генерише реченице које представљају опис задате слике

3. АРХИТЕКТУРА

3.1. Енкодер–декодер

Архитектура имплементираног система за аутоматско генерисање описа слика заснива се на комбинацији два типа неуронских мрежа: конволуционих неуронских мрежа (CNN), које служе за екстракцију визуелних карактеристика слике, и рекурентних мрежа (RNN), које се користе за генерисање текстуалног описа. Ове две неуронске мреже у систему чине два главна модула: енкодер слике и декодер текста. Овакав модел постао је стандардни приступ у задацима који укључују трансформацију једне врсте података у другу.

Улога енкодера је да улазну слику трансформише у богату векторску репрезентацију фиксне дужине. У овом случају, VGG-16 модел обезбеђује вектор димензије 4096, који сажима кључне визуелне карактеристике слике. Овај вектор се затим пројектује у просто мање димензије помоћу потпуно повезаног слоја и прослеђује LSTM мрежи.

Декодер користи ову репрезентацију слике као иницијални улаз за генерисање реченице. Процес генерисања је итеративан. На основу репрезентације слике и претходно генерисаних речи, LSTM предвиђа следећу реч у низу. Овај поступак се понавља све док се не добије специјални токен за крај реченице.

Архитектура енкодер-декодер модела омогућава да се визуелне информације из слике директно повежу са лингвистичким знањем уграђеним у LSTM. На овај начин се остварује мост између рачунарског вида и обраде природног језика.

3.2. Енкодер VGG-16 неуронска мрежа

За енкодер слике користи се унапред истренирана конволуциона мрежа VGG-16, која је обучена на великом скупу података *ImageNet*. Принцип коришћења претходно тренираних модела се често примењује у пракси и назива се пренесено учење (енгл. *Transfer Learning*). То је техника у којој се знање стечено решавањем једног задатка користи за брже и ефикасније решавање другог, сродног задатка. Уместо да се модел тренира од почетка, и то са насумично иницијализованим параметрима, полази се од већ тренираног модела на великом скупу података, а затим се тај модел прилагођава (*fine-tuning*) на мањем, специфичном скупу података који одговара конкретном проблему. Код класификације слика, карактеристике ниског нивоа као што су ивице, облици и осветљење могу бити заједничке за више задатака. Преносом тог знања, модел се много брже и ефикасније прилагођава новој области, чак и са мало података.

CNN је задужена да улазну слику трансформише у богату репрезентацију фиксне дужине. Уместо да буде искоришћен последњи слој намењен класификацији, архитектура се сече тако да излаз долази из претпоследњег потпуно повезаног слоја (fc2). Овај слој генерише димензије 4096, који сажима кључне карактеристике слике.

VGG-16 конволуциона неуронска мрежа представљена је 2014. године од стране *Karen Simonyan* и *Andrew Zisserman*-а у њиховом раду [4]. Они су у том раду нагласили да је дубина неуронске мреже један од најважнијих аспеката приликом дизајнирања архитектуре, на основу чега су повећали дубину додавањем

конволуционих слојева мањих димензија. Ова сазнања су применили на *ImageNet* такмичењу 2014. године на коме су освојили прво место за задатак за локализацију, и друго место за задатак за класификацију. На овај начин дошли су до побољшања модела уз смањење броја параметара. Резултат је био прилично прецизнија конволуциона мрежа која успешно може бити примењена и на другим скуповима података са сликама.

Мрежа се састоји из пет конволуционих блокова (укупно 13 конволуционих слојева) и три потпуно повезана слоја (енгл. *Fully Connected Layers* - FC): прва два имају по 4096 канала, а трећи слој врши класификацију за 1000 класа и стога има 1000 канала, односно по један канал за сваку класу. Последњи слој је *softmax* слој. Управо због овакве конфигурације са 13 конволуционих слојева и 3 FC слоја мрежа је добила назив VGG-16. Постоји и варијанта VGG-19 која садржи 16 CNN слојева и 3 FC слоја.

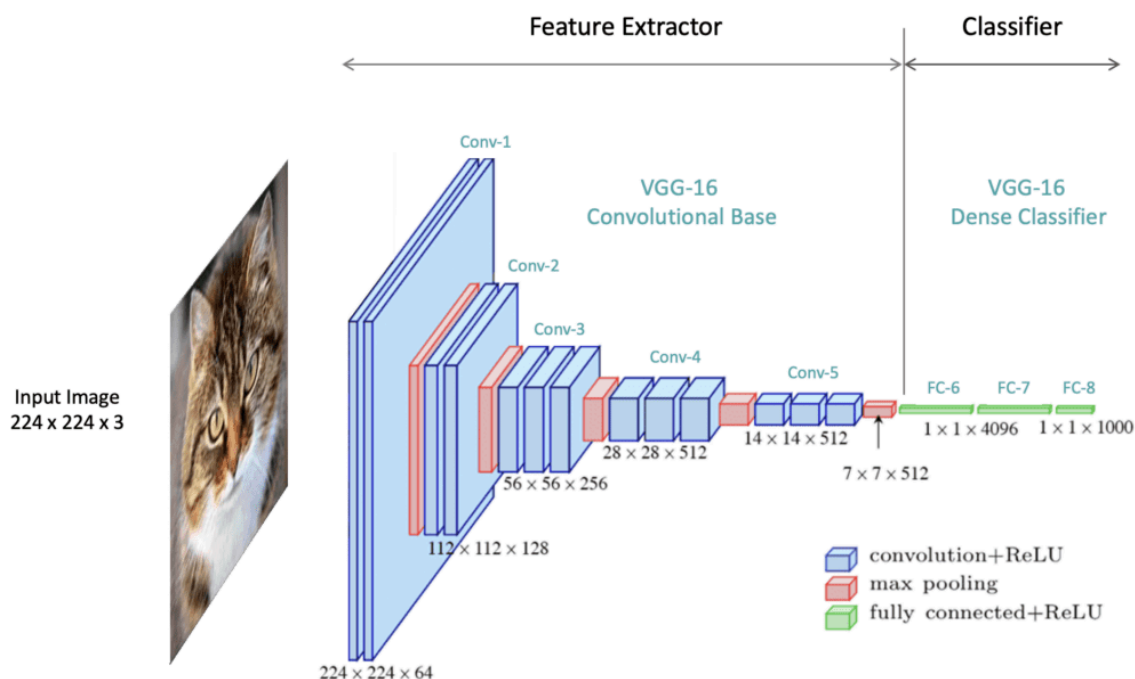
Карактеристично за VGG-16 мрежу (и друге варијанте VGG мрежа) је то што се користе конволуциони слојеви малих филтера величине 3x3 уместо великих филтера (нпр. 11x11 или 7x7). Изабрана је димензија 3x3 јер је најмања величина која обухвата појмове центар, лево/десно, горе/доле [4]. Више узастопних 3x3 конволуција могу ефикасно имитирати већи филтер, али са мање параметара и већом нелинеарношћу. Као активациона функција користи се ReLU функција након сваке конволуције и FC слоја. Корак приликом конволуције је фиксан и износи 1 пиксел.

Попуњавање простора (*padding*) улаза конволуционог слоја је такво да се просторна резолуција очува након конволуције, односно у овом случају за филтер 3x3 филтер *padding* износи 1 пиксел.

Модел почиње са пет конволуционих блокова који чине део за екстракцију карактеристика (енгл. *Feature Extractor*) [3]. Конволуциони блок је општи термин који се користи за опис низа слојева унутар конволуционе неуронске мреже, који се често понављају у делу за екстракцију карактеристика. Први део за екстракцију карактеристика задужен је за препознавање и извлачење важних визуелних карактеристика из улазне слике. Након што су карактеристике извучене, оне се прослеђују класификатору. Његов задатак је да те карактеристике претвори у предвиђања класа. У случају VGG-16 модела, који је трениран на *ImageNet* скупу података, излазни слој садржи 1000 неурона. Вредности ових неурона представљају вероватноће да улазна слика припада једној од 1000 класа. Предвиђена класа је она са највећом вероватноћом. Распоред ових слојева и илустрација архитектуре VGG-16 мреже приказана је на слици 2:

- Први блок се састоји од два конволуциона слоја која имају 64 филтра величине 3x3. Након што се улазна RGB слика димензија 224x224x3 (дубина је 3, једна за сваки канал) пропусти кроз прва два конволуциона слоја, добија се промена димензија на 224x224x64. Након тога следи *max pooling* слој димензија 2x2 са кораком (*stride*) 2;
- Други блок се састоји такође од два конволуциона слоја, с тим што они сада садрже по 128 филтра величине 3x3. Иза њих такође следи 2x2 *max pooling* слој са кораком од 2, након чега се димензије смањују на 56x56x128;
- Трећи блок садржи три конволуциона слоја, са филтерима величине 3x3. Сваки од њих користи 256 филтра. Ови слојеви су, као и претходна два, праћени 2x2 *max pooling* слојем са кораком 2;

- У четвртном и петом блоку се налазе по три конволуциона слоја са 512 филтра величине 3×3 . Након оба блока примењује се *max pooling* слој димензије 2×2 са кораком 2;
- Након ових пет блокова налазе се два потпуно повезана слоја од којих сваки има по 4096 неурона;
- Последњи слој је излазни слој са *softmax* активационом функцијом који даје излаз класификације. У овом случају то је 1000 класа за *ImageNet*.



Слика 2. Архитектура VGG-16 конволуционе неуронске мреже

3.3. Декодер - LSTM рекурентна неуронска мрежа

Декодер у систему служи за генерисање описа слике на основу векторске репрезентације коју производи енкодер. За ову сврху се користи рекурентна неуронска мрежа у виду *Long Short-Term Memory* (LSTM) архитектуре. LSTM је специјализована врста RNN мреже која је способна да памти и користи дугорочне зависности у секвенцама података, што је од кључног значаја код генерисања течних и смислених реченица.

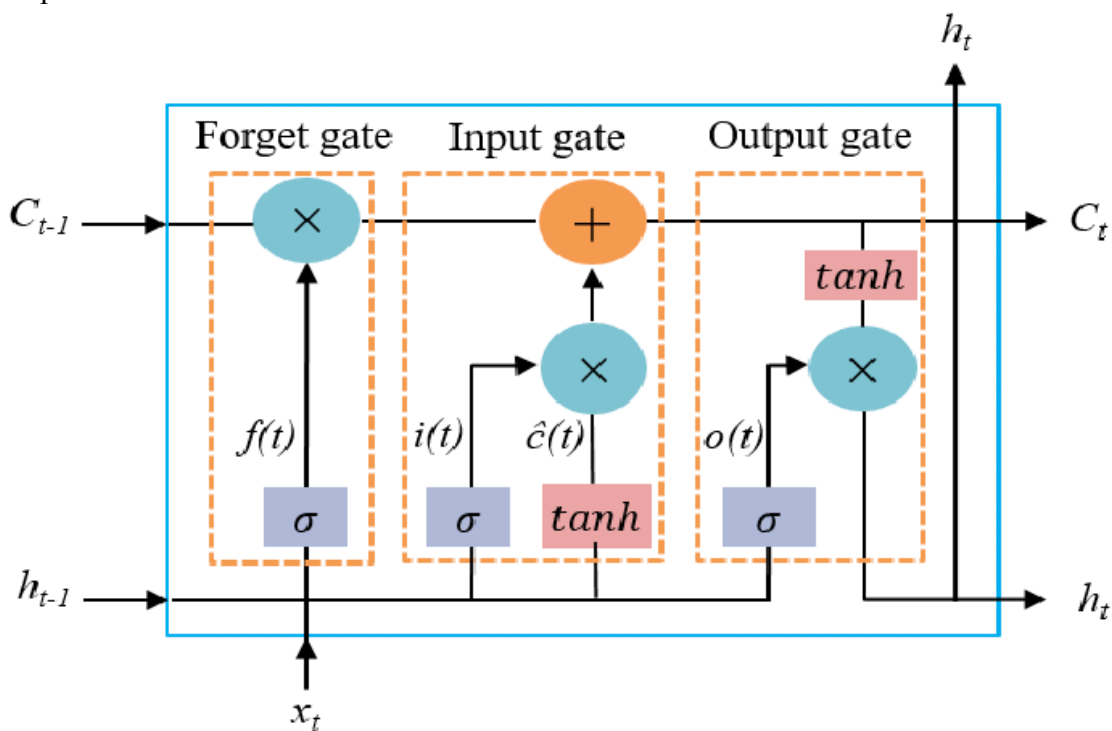
LSTM мрежа је први пут представљена 1997. године у раду „*Long Short-Term Memory*“ аутора *Hochreiter*-а и *Schmidhuber*-а [5]. Они су нагласили ограничења која имају традиционалне рекурентне неуронске мреже (RNN) у процесу учења дугорочних зависности у секвенцијалним подацима. Традиционалне рекурентне неуронске мреже често имају проблем недостајућег градијента, где градијенти који се користе за обуку могу постати премали, што омета способност мреже да учи из ранијих временских корака у секвенци. LSTM мреже су посебно дизајниране да превазиђу овај изазов, омогућавајући моделирање секвенци где је контекст ранијих

улаза кључан за тачна предвиђања. Сам назив ове мреже одражава њен основни циљ: мрежа дугорочне меморије.

Мотивација за креирање LSTM мрежа била је потреба за архитектуром која би могла да прати информације током дужег временског периода, а да притом ефикасно заборавља небитне податке. Ова могућност је посебно важна у задацима као што су моделирање језика, препознавање говора и описивање слика, где контекст који пружају претходни улаз значајно утиче на излаз. LSTM архитектура укључује механизме зване капије (енгл. *gates*), које контролишу ток информација, омогућавајући мрежи да ефикасно оджава и ажурира своју меморију.

3.3.1. Архитектура LSTM мрежа

Једна од кључних компоненти LSTM мреже је стање ћелије које се понаша као меморија LSTM-а, преносећи информације кроз целу секвенцу. Архитектура LSTM ћелије се разликује од традиционалних RNN ћелија управо због раније поменутог механизма капија, које регулишу ток информација. Слика 3 приказује архитектуру LSTM јединице која се састоји од меморијске ћелије и три врсте капија: улазна капија (енгл. *input gate*), капија заборављања (енгл. *forget gate*) и излазна капија (енгл. *output gate*). Меморијска ћелија памти вредности током произвољних временских интервала.



Слика 3. Архитектура LSTM јединице [6]

Капија заборављања се користи за одлучивање која информација из претходне ћелије C_{t-1} треба да се одбаца. Она такође користи сигмоидалну активациону функцију (σ) за добијање вредности од 0 до 1. Излаз сигмоидалне функције је $f(t)$.

Уколико излаз има вредност 0, то значи да се информација потпуно заборавља, док 1 значи да се информација прослеђује даље кроз модел. Улаз у сигмоидалну функцију је тренутни улаз x_t и претходно скривено стање h_{t-1} .

Улазна капија контролише меру у којој се додају нове информације у стање ћелије. Узима претходно скривено стање h_{t-1} и тренутни улаз x_t и примењује сигмоидалну активациону функцију (σ) за одлучивање које вредности, тј. које информације треба ажурирати. Излаз ове функције $i(t)$ се множи са $\tanh$ активационом функцијом која на излазу даје ново стање ћелије $\hat{c}(t)$. Резултат овог множења се додаје тренутном стању ћелије C_t .

Излазна капија, слично осталим капијама, користи сигмоидалну функцију за одлучивање који део стања ћелије треба да се проследи следећем слоју. Стање ћелије се трансформише помоћу $\tanh$ да би се добиле вредности између -1 и 1. Затим су те вредности помножене излазом сигмоидалне функције $o(t)$ како би се добиле излазне вредности, где је h_t ново скривено стање.

Интеракција ових капија омогућава LSTM мрежама да одржавају и манипулишу стањем ћелије током времена, ефикасно ублажавајући проблем нестајућих градијената који често представља проблем код традиционалних рекурентних неуронских мрежа. Овај проблем се решава захваљујући томе што LSTM омогућава скоро непромењен пренос информација, па самим тим и градијената, кроз ћелијско стање C_t . То је омогућено захваљујући архитектури која је претходно описана. Током тренирања, градијенти се могу стабилно преносити уназад захваљујући начину на који се стање ћелије ажурира. Пошто капије дозвољавају да се информације преносе без драстичних промена, спречава се нагло смањење или нестајање градијената током пропагације. Ова способност чини LSTM посебно ефикасним у задацима где је од значаја задржати информације из ранијих временских корака и омогућава ефикасно моделовање дугих секвенци, што је неопходно код генерисања реченица које садрже више од неколико речи [5].

3.3.2. Примена LSTM у систему за генерисање описа слика

У систему за генерисање описа слика, улаз у декодер је вектор димензије 4096 који производи VGG-16 енкодер. Пре него што се проследи LSTM мрежи, овај вектор се трансформише у мању димензију помоћу потпуно повезаног слоја, како би био компатибилан са димензијом улаза LSTM ћелија.

Процес генерисања описа функционише секвенцијално, реч по реч. На почетку, LSTM као улаз добија репрезентацију слике и специјални токен за почетак реченице ($\langle start \rangle$). Затим мрежа итеративно генерише следеће речи на основу претходних речи и унутрашњег скривеног стања. Генерисање се завршава када се произведе специјални токен за крај реченице ($\langle end \rangle$).

Тренинг декодера се заснива на алгоритму који се назива *teacher forcing*. Током учења, уместо предвиђених речи моделу се прослеђују стварне речи из скупа података. На тај начин се убрзава учење и постиже боља стабилност. Губитак се рачуна као укупан негативни губитак логаритамске вероватноће (енгл. *Negative Log-Likelihood Loss*) између предвиђених и стварних речи у секвенци. За декодер се могу користити и друге варијанте рекурентних мрежа, као што су GRU (*Gated Recurrent Unit*), али LSTM је најчешће биран због своје робусности и бољих

перформанси у задацима секвенцијалног моделовања. Као што су аутори закључили у свом раду [5], LSTM се показао знатно ефикаснијим од обичних RNN мрежа у задацима предвиђања и класификације секвенци, посебно тамо где је важно дугорочно памћење.

3.4. Евалуација модела

За евалуацију су коришћене BLEU-1 и BLEU-2 метрике. Оне су широко коришћене за утврђивање квалитета генерисаних описа поређујући их са њиховим референтним описима. BLEU (енгл. *Bilingual Evaluation Understudy*) метрика се заснива на поређењу n -грама кандидата и n -грама референтних превода, при чему се броје поклапања независно од њихове позиције у реченици. Већи број поклапања указује на виши квалитет превода.

Резултати BLEU евалуације крећу се у интервалу од 0 до 1. Максимална вредност 1 може се добити само уколико се превод потпуно подудара са референтним преводом, што се у пракси ретко дешава. Чак и људски преводиоци не постижу увек максималан резултат. Важан аспект методе јесте да већи број референтних превода за исту реченицу повећава остварени резултат, због чега се мора водити рачуна приликом поређења евалуација са различитим бројем референци. На пример, у тексту који је обухватио око 500 реченица из новинских чланака, људски преводилац је остварио BLEU скор 0.3468 у поређењу са четири референтна превода, док је исти превод, када су коришћене само две референце, имао скор 0.2571 [7].

BLEU-1 мери преклапање појединачних речи, односно мери слагање на нивоу униграма ($n=1$), док BLEU-2 мери преклапање парова узастопних речи (мери на основу биграма). Ове метрике дају квантитативну меру квалитета генерисаних наслова. Високе вредности BLEU-а (ближе 1.0) указују на бољи квалитет генерисаног текста.

4. *PROOF-OF-CONCEPT* ПРОЈЕКАТ

4.1. Коришћене технологије

За имплементацију система коришћене су следеће технологије и библиотеке:

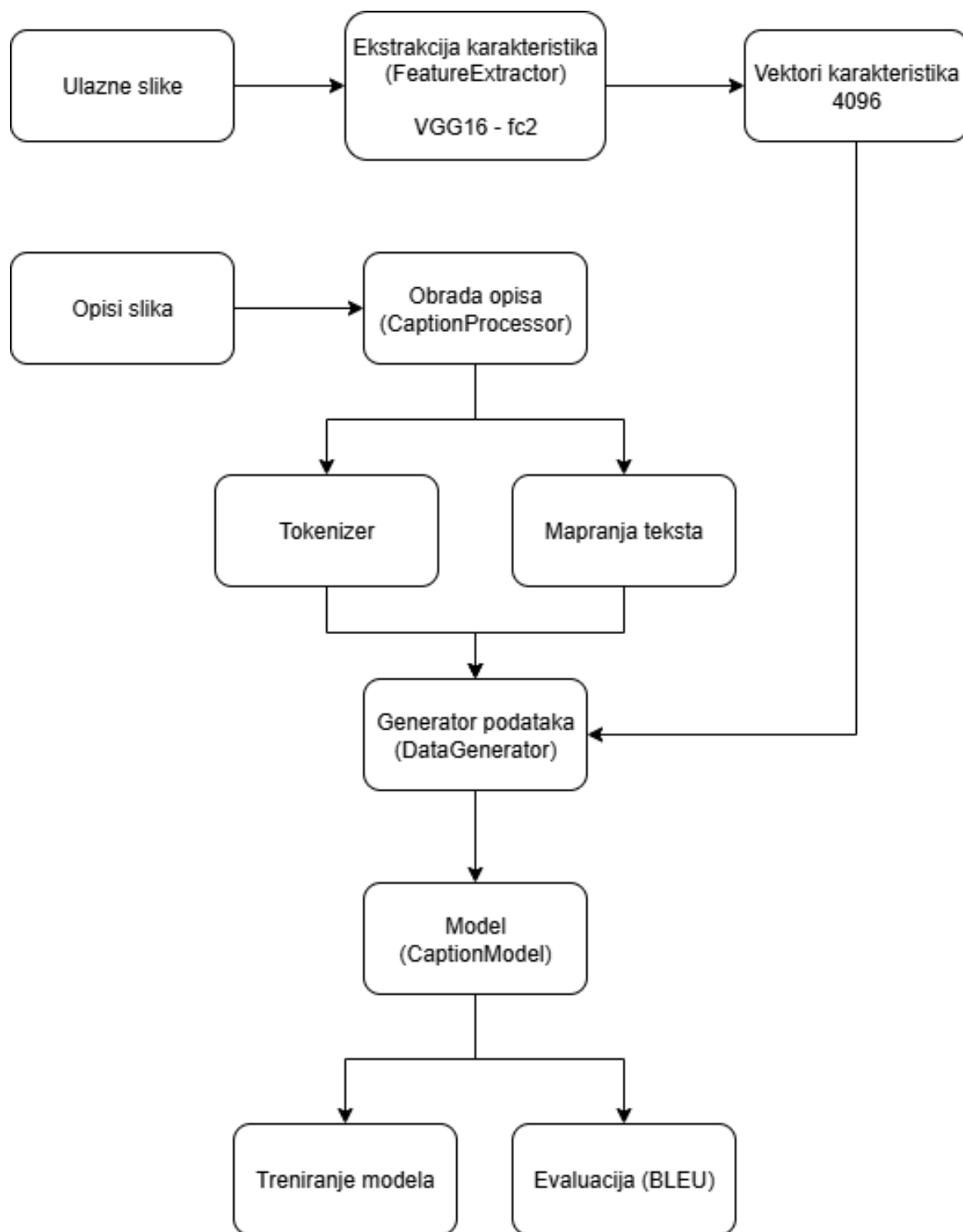
- Python: програмски језик који се широко користи у машинском учењу;
- TensorFlow/Keras: главни *framework* за рад са дубоким учењем. Ово је главна библиотека за дубоко учење која се користи за изградњу, тренирање и евалуацију модела. Нуди API високог нивоа (*Keras*) који поједностављује развој неуронских мрежа;
- NumPy: библиотека за рад са низовима и матрицама;
- Matplotlib: библиотека за визуализацију резултата;
- PIL: библиотека за рад са сликама;
- NLTK: користи се за евалуацију модела помоћу BLEU метрике, која упоређује машински генерисан текст са референтним текстом;
- tqdm: библиотека за *progress bar*;
- Tokenizer: компонента из *Keras* библиотеке која се користи за претварање текста (налова слика) у нумеричке секвенце. Она ствара речник (*vocab*) и мапира сваку реч на јединствени цео број, што је неопходно за обраду текста у неуронским мрежама;
- Pickle: *Python* модул за серијализацију и десеријализацију објеката. У коду се користи за чување и учитавање обрађених података (као што су мапирање слика и наслова, и *tokenizer*) како би се избегло поновно процесирање.

4.2. Архитектура апликације

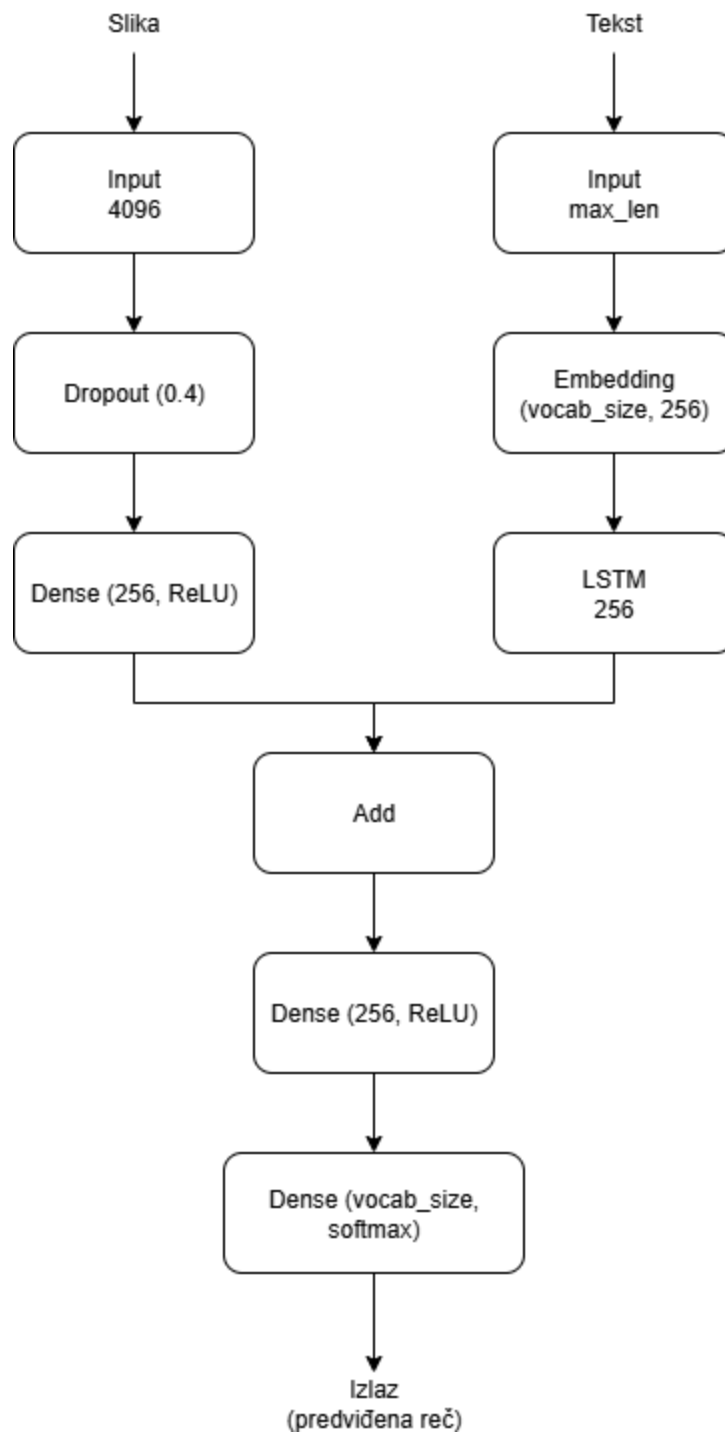
Структура апликације се састоји из неколико главних компоненти које функционишу заједно како би се остварио циљ *Image Captioning*-а коришћењем енкодер-декодер принципа:

- *caption_processor.py*: служи за учитавање и чишћење описа, као и за токенизацију;
- *feature_extraction.py*: користи се за екстракцију визуелних карактеристика из слика;
- *data_generator.py*: креирање генератора података за тренирање;
- *model_builder.py*: дефинисање архитектуре модела;
- *main.py*: иницијализација потребних компоненти и њихово оркестрирање током тренирања, као и евалуација модела;
- *demo.py*: покретање модела на новим сликама;
- *utils.py*: фајл у коме су смештене помоћне функције (нпр. за предикцију и евалуацију).

Архитектура апликације приказана је на слици 4, док је модел приказан на слици 5.



Слика 4. Архитектура система за Image Captioning



Слика 5. Модел за Image Captioning (CaptionModelBuilder)

4.3. Екстракција карактеристика из слика (*feature extraction*)

Ова компонента (класа *FeatureExtractor*) је одговорна за претварање слика у нумеричке векторе. За то се користи модел конволуционе неуронске мреже VGG-16 из библиотеке *Keras* претрениран на *ImageNet*-у, као што је раније речено. Уместо да се користи последњи излаз VGG-16 мреже, користи се излаз из претпоследњег слоја (fc2) димензије 4096. То је постигнуто у тринаестој линији кода приказаног на слици 6, где се као *outputs* дефинише *layers[-2]*, што је претпоследњи елемент у низу слојева. Функција *image_features_extraction* из фајла *utils.py* учитава слику, конвертује у улаз димензије 224x224, нормализује, односно конвертује је у низ и прослеђује конволуционој неуронској мрежи. Свака слика се обрађује и добија нумеричку репрезентацију (вектор фиксне димензије 4096) која представља њене визуелне карактеристике. Ови вектори се чувају у *features.pkl* фајлу.

```

10 class FeatureExtractor:
11     def __init__(self, features_file, images_directory):
12         vgg_model = VGG16()
13         self.model = Model(inputs = vgg_model.inputs, outputs = vgg_model.layers[-2].output)
14         self.features_file = features_file
15         self.images_directory = images_directory
16
17     def extract(self):
18         features = {}
19         for img_name in tqdm(os.listdir(self.images_directory)):
20             img_id = img_name.split('.')[0]
21             img_path = os.path.join(self.images_directory, img_name)
22             features[img_id] = image_features_extraction(img_path, self.model)
23
24         with open(self.features_file, "wb") as f:
25             pickle.dump(features, f)
26
27         with open(self.features_file, "rb") as f:
28             features = pickle.load(f)
29
30         return features

```

Слика 6. Класа *FeatureExtractor* за извлачење карактеристика слика

```

30 def image_features_extraction(img_path, model):
31     img = load_img(img_path, target_size=(224, 224))
32     img = img_to_array(img)
33     img = img.reshape((1, img.shape[0], img.shape[1], img.shape[2]))
34     img = preprocess_input(img)
35     features = model.predict(img, verbose=0)
36
37     return features

```

Слика 7. Помоћна функција за припремање слике која се прослеђује CNN

4.4. Обрада наслова (*Caption Processor*)

Класа *CaptionProcessor* (слика 8) је задужена за обраду и чишћење текстуалних наслова. Подразумева следеће кораке: учитавање наслова, мапирање сваког наслова на одговарајућу слику, чишћење текста (уклањање специјалних знакова, претварање у мала слова, итд.) и додавање специјалних токена *startseq* и *endseq* како би се означио почетак и крај реченице. Све то се ради позивањем функције *clean_captions*, која најпре позива *build_mapping* за мапирање наслова на одговарајућу слику, а затим позива *clean* за чишћење текста. На крају, ова компонента у функцији *prepare_tokenizer* креира *Tokenizer* који мапира речи на индексе и чува га у *tokenizer.pkl* фајлу. Ово омогућава моделу да зна када да започне и заврши генерисање наслова. Функција *pad_sequences* у фајлу *data_generator.py* осигурава да све улазне секвенце имају исту дужину.

```

6   class CaptionProcessor:
52     def clean_captions(self):
53         self.build_mapping()
54         self.clean()
55
56         return self.mapping
57
58     def prepare_tokenizer(self):
59         all_captions = []
60         for key in self.mapping:
61             for caption in self.mapping[key]:
62                 all_captions.append(caption)
63
64         tokenizer = Tokenizer()
65         tokenizer.fit_on_texts(all_captions)
66         vocab_size = len(tokenizer.word_index) + 1
67         max_len = max([len(caption.split()) for caption in all_captions])
68
69         with open(self.tokenizer_file, "wb") as f:
70             pickle.dump(tokenizer, f)
71
72         return tokenizer, vocab_size, max_len
73

```

Слика 8. Главне функције класе за обраду и чување наслова

4.5. Генератор података (*Data Generator*)

Класа *DataGenerator* креира пакете података (*batch*) за тренирање модела. Ова компонента је кључна јер динамички генерише улазне и излазне секвенце, претварајући наслове у парове (*input/output*) из модела. На пример, за наслов „*startseq a dog is walking endseq*“, генератор ствара нове парове: („*startseq*“, „*a*“), („*startseq a*“, „*dog*“), („*startseq a dog*“, „*is*“), итд.

```

5  class DataGenerator:
14     def data_generator(self, data_keys):
15         n = 0
16         X1, X2, Y = list(), list(), list()
17         while True:
18             for key in data_keys:
19                 n = n + 1
20                 for caption in self.mapping[key]:
21                     sequence = self.tokenizer.texts_to_sequences([caption])[0]
22                     for i in range(1, len(sequence)):
23                         input_seq, out_seq = sequence[:i], sequence[i]
24                         input_seq = pad_sequences([input_seq], maxlen = self.max_len, padding = 'post')[0]
25                         out_seq = to_categorical([out_seq], num_classes = self.vocab_size)[0]
26                         X1.append(self.features[key][0])
27                         X2.append(input_seq)
28                         Y.append(out_seq)
29
30                 if n == self.batch_size:
31                     n = 0
32                     X1, X2, Y = np.array(X1), np.array(X2), np.array(Y)
33                     yield (X1, X2), Y
34
35                 X1, X2, Y = list(), list(), list()
36
37     def generator(self, data_keys):
38         return self.data_generator(data_keys)

```

Слика 9. Имплементација класе *DataGenerator*

4.6. Модел за генерисање наслова (*Caption Model*)

Ова компонента (*CaptionModelBuilder*) смештена у фајлу *model_builder.py* представља главни модел за учење, слика 10.

Модел се састоји из две гране:

- Грана 1 (слика): Први улаз (слика) има облик (4096, 1) који представља векторске карактеристике слике из VGG-16. Након проласка кроз *Dense* слој, добија се вектор димензија 256.

$$\text{Input}(4096) \rightarrow \text{Dropout}(0.4) \rightarrow \text{Dense}(256, \text{ReLU})$$

- Грана 2 (текст): Други улаз (текст, односно текстуалне секвенце из *Tokenizer*-а) се обрађује *Embedding* слојем који претвара индексе речи у густе (енгл. *dense*) векторе.

$$\text{Input}(\text{max_len}) \rightarrow \text{Embedding}(\text{vocab_size}, 256) \rightarrow \text{LSTM}(256)$$

- Спајање: Слој *add* комбинује излаз *Dense* слоја (визуелне карактеристике) и излаз *LSTM* слоја (текстуалне карактеристике), што омогућава моделу да истовремено обрађује информације о слици и до сада генерисаном тексту. Излазни слој има *softmax* активациону функцију, што је идеално за проблеме класификације речи, где се предвиђа вероватноћа за сваку реч у речнику.

$$\text{Сабирање обе гране} \rightarrow \text{Dense}(256, \text{ReLU}) \rightarrow \text{Dense}(\text{vocab_size}, \text{softmax})$$

```

3  class CaptionModelBuilder:
4      def __init__(self, vocab_size, max_len):
5          self.vocab_size = vocab_size
6          self.max_len = max_len
7
8      def build(self):
9          inputs1 = Input(shape = (4096,))
10         fe1 = Dropout(0.4)(inputs1)
11         fe2 = Dense(256, activation='relu')(fe1)
12
13         inputs2 = Input(shape = (self.max_len,))
14         se1 = Embedding(self.vocab_size, 256, mask_zero = True)(inputs2)
15         se2 = Dropout(0.4)(se1)
16         se3 = LSTM(256)(se2)
17
18         decoder1 = add([fe2, se3])
19         decoder2 = Dense(256, activation = 'relu')(decoder1)
20         outputs = Dense(self.vocab_size, activation = 'softmax')(decoder2)
21
22         model = Model(inputs = [inputs1, inputs2], outputs = outputs)
23         model.compile(loss = 'categorical_crossentropy', optimizer = 'adam')
24
25         return model

```

Слика 10. Главни модел за генерисање описа слика

Текст се генерише тако што LSTM мрежа прима претходне речи из описа. У комбинацији са векторском репрезентацијом слике, предвиђа следећу реч.

4.7. Главни програм (*Main*)

Скрипта *main.py* интегрише све компоненте система за генерисање описа слика. На самом почетку се иницијализују класе *FeatureExtractor* и *CaptionProcessor*, задужене за екстракцију визуелних карактеристика из слика и обраду текстуалних описа. Карактеристике се добијају применом претходно тренираног конволуционог модела и чувају у бинарном фајлу *features.pkl*, док се описи (*caption*-и) чисте, нормализују и претварају у нумеричку форму помоћу токенизатора. Након тога, подаци се деле у размери 90:10 на тренирајући и тест скуп, при чему се из тренирајућег скупа издваја 10% за валидацију. Приказано је на слици 11.

```
16 def main_with_checkpoint():
17     extractor = FeatureExtractor(
18         os.path.join(Config.WORKING_DIR, "features.pkl"),
19         os.path.join(Config.BASE_DIR, "Images")
20     )
21     features = extractor.extract()
22
23     processor = CaptionProcessor(
24         os.path.join(Config.BASE_DIR, "captions.txt"),
25         os.path.join(Config.WORKING_DIR, "mapping.pkl"),
26         os.path.join(Config.WORKING_DIR, "tokenizer.pkl")
27     )
28     mapping = processor.clean_captions()
29     tokenizer, vocab_size, max_len = processor.prepare_tokenizer()
30
31     with open(os.path.join(Config.WORKING_DIR, "max_len.txt"), "w") as f:
32         f.write(str(max_len))
33
34     image_ids = list(mapping.keys())
35     split = int(len(image_ids) * 0.9)
36     train_ids = image_ids[:split]
37     test_ids = image_ids[split:]
38
39     val_split = int(len(train_ids) * 0.9)
40     val_ids = train_ids[val_split:]
41     train_ids = train_ids[:val_split]
```

Слика 11. Иницијализација екстракције карактеристика и обраде описа

За креирање улазних података користи се класа *DataGenerator*, која динамички генерише парове података у облику карактеристика слике и делимичног описа, док је циљна вредност следећа реч у секвенци, слика 12. Ови подаци се затим пакују у *TensorFlow tf.data.Dataset* структуру ради ефикасног учитавања током тренинга. Модел се креира помоћу класе *CaptionModelBuilder*, која враћа архитектуру засновану на комбинацији конволуционе (CNN) и рекурентне (LSTM) мреже.

```

43 data_gen = DataGenerator(mapping, features, tokenizer, max_len, vocab_size, Config.BATCH_SIZE)
44 dataset = tf.data.Dataset.from_generator(
45     lambda: data_gen.generator(train_ids),
46     output_signature=(
47         (
48             tf.TensorSpec(shape = (None, 4096), dtype = tf.float32), # X1
49             tf.TensorSpec(shape = (None, max_len), dtype = tf.int32) # X2
50         ),
51         tf.TensorSpec(shape = (None, vocab_size), dtype = tf.float32) # Y
52     )
53 )
54
55 val_dataset = tf.data.Dataset.from_generator(
56     lambda: data_gen.generator(val_ids),
57     output_signature=(
58         (
59             tf.TensorSpec(shape=(None, 4096), dtype=tf.float32), # X1
60             tf.TensorSpec(shape=(None, max_len), dtype=tf.int32) # X2
61         ),
62         tf.TensorSpec(shape=(None, vocab_size), dtype=tf.float32) # Y
63     )
64 )
65
66 builder = CaptionModelBuilder(vocab_size, max_len)
67 model = builder.build()

```

Слика 12. Креирање скупова података и иницијализација модела

Тренирање модела се врши позивом методе *fit*, са параметрима дефинисаним у фајлу *config.py* (EPOCHS = 50, BATCH_SIZE = 64). Током тренинга се користи функција губитка *categorical_crossentropy* и оптимизатор *Adam*, док се напредак прати преко валидационог губитка (*val_loss*). Укључен је механизам за чување најбољег модела помоћу *callback* функције *ModelCheckpoint*, која на свакој епохи снима модел ако је постигнут бољи резултат, слика 13.

```

69 checkpoint_path = os.path.join(Config.MODELS_DIR, "caption_model_checkpoint.keras")
70
71 checkpoint = tf.keras.callbacks.ModelCheckpoint(
72     filepath=checkpoint_path,
73     monitor='val_loss',
74     save_best_only=True,
75     mode='min',
76     verbose=1
77 )
78
79 model.fit(
80     dataset,
81     validation_data=val_dataset,
82     epochs=Config.EPOCHS,
83     steps_per_epoch=len(train_ids) // Config.BATCH_SIZE,
84     validation_steps=len(val_ids) // Config.BATCH_SIZE,
85     callbacks=[checkpoint],
86     verbose=1
87 )
88
89 best_model = tf.keras.models.load_model(checkpoint_path)

```

Слика 13. Тренирање модела са *checkpoint*-ом

Након завршетка тренинга, најбољи модел се учитава и врши се тестирање на засебном тест скупу приказано на слици 14. За сваку слику генерише се предвиђени опис коришћењем функције *predict_caption*, након чега се резултати оцењују помоћу BLEU метрике (BLEU-1 и BLEU-2), која мери степен поклапања између стварних и генерисаних описа. На овај начин се добија квантитативна процена квалитета модела у задатку аутоматског генерисања описа слика.

```
90     # test
91     actual,predicted=list(), list()
92     for key in tqdm(test_ids):
93         captions = mapping[key]
94         y_pred = predict_caption(best_model, features[key], tokenizer, max_len)
95
96         actual_captions = [caption.split() for caption in captions]
97         y_pred = y_pred.split()
98         actual.append(actual_captions)
99         predicted.append(y_pred)
100
101     print("BLEU-1: %f"% corpus_bleu(actual, predicted, weights = (1.0,0,0,0)))
102     print("BLEU-2: %f"% corpus_bleu(actual, predicted, weights = (0.5,0.5,0,0)))
```

Слика 14. Тестирање

5. ПРЕГЛЕД РЕЗУЛТАТА

5.1. Анализа грешака

Модел је у значајном делу случајева успео да генерише садржајно адекватне и граматички исправне описе, који на релативно прецизан начин одражавају визуелни садржај приказан на сликама. Генерисани описи коректно препознају главне објекте, радње и односе у сцени, што указује на способност модела да успешно повеже визуелне карактеристике са одговарајућим језичким репрезентацијама. Неки од примера адекватних описа су приказани на слици 15.



Слика 15. Пример адекватних описа слика

Ипак, у доста примера модел није успео да генерише потпуно тачне описе, слика 16. Најчешће грешке односе се на погрешно препознавање пола особа, нетачан број објеката у сцени, као и на погрешно навођење боја појединих елемената. На пример, у ситуацијама где је на слици приказана жена, модел је понекад генерисао опис у коме се помиње мушка особа, што указује на ограничену способност разликовања полних карактеристика у визуелним подацима. Такође, у случајевима где се у сцени налази више ентитета (нпр. више животиња или особа), модел је повремено давао описе у једнини, што показује да има потешкоћа у процени броја објеката. Слично томе, грешке у описивању боја - као што су замене између „плаве“ и „зелене“ или између сличних нијанси - указују на ограничену прецизност у визуелној перцепцији модела. Ове појаве су очекиване код модела ове врсте и могу се додатно унапредити проширивањем скупа података и дужим процесом тренирања.



Слика 16. Пример непотпуно тачних описа слика

Поред мањих нетачности у описима, у појединим случајевима модел је правио и веће грешке у интерпретацији слике (слика 17). На пример произвољно навођење неких објеката, што указује на ограничену способност модела да правилно идентификује главне ентитете на сцени. Такође, забележени су примери где модел на почетку производи релативно тачан опис, али се „збуну“ приликом описивања боја или детаља, понављајући једну или више боја више пута и не успевајући да доврши потпуну реченицу. Ове грешке илуструју слабости модела у задржавању контекста током генерисања, што представља типичан изазов код генеративних модела за аутоматско описивање слика.



Слика 17. Пример погрешних описа слика

5.2. Поређење стратегија

Модел је иницијално трениран у конфигурацији која користи чување најбоље верзије модела (*checkpoint* механизам) на основу најнижег валидационог губитка (*val loss*). Овај приступ се показао као најстабилнији, јер омогућава да се задржи модел који постиже најбољи резултат током тренирања и спречава проблем претренираности.

Поред тога, спроведени су и експерименти са различитим стратегијама тренирања - модел без памћења најбољих тежина (без *checkpoint* механизма) и модел са применом раног заустављања (*EarlyStopping*) на основу валидационог губитка. Међутим, ови приступи су дали нешто слабије резултате.

Према израчунатим BLEU метрикама, модел са *checkpoint* механизмом је постигао највише вредности (BLEU-1: 0.500580, BLEU-2: 0.300099), што указује на бољу способност генерисања описа који су семантички и синтаксички ближи стварним описима из скупа података.

С друге стране, код модела без чувања најбољих тежина (BLEU-1: 0.485922, BLEU-2: 0.265095) и модела са раним заустављањем (BLEU-1: 0.460417, BLEU-2: 0.276025) примећен је благи пад BLEU-1 и BLEU-2 скорова, што указује на недовољну стабилност током тренинга или на прерано прекидање процеса учења.

6. ЗАКЉУЧАК

У овом раду реализован је *proof-of-concept* систем за аутоматско генерисање описа слика (*Image Captioning*) који комбинује методе рачунарског вида и обраде природног језика. Основна идеја била је да се примени *encoder-decoder* архитектура која омогућава повезивање визуелних и језичких репрезентација података.

У делу истраживања који се односи на екстракцију визуелних карактеристика, примењене су конволуционе неуронске мреже (CNN), које су успешно идентификовале и кодовале релевантне објекте и сцене са слика у векторски простор. Овај корак представља основу за разумевање садржаја и његову даљу обраду у језичком домену.

За генерисање текстуалних описа примењене су рекурентне неуронске мреже (RNN) са LSTM јединицама, које су омогућиле генерисање кохерентних и семантички смислених реченица. Модел је показао да може да научи повезивање визуелних шаблона са језичким структурама, што је потврђено квалитативним примерима и метриком BLEU, која је коришћена за процену резултата.

Циљ рада је развој система који интегрише технике дубоког учења ради разумевања и описивања слика. Имплементација је потврдила да комбинација CNN и LSTM мрежа може дати солидне резултате у задацима мултимодалног учења, иако је у питању почетна, концептуална верзија модела.

Током реализације, показано је да квалитет резултата значајно зависи од избора скупа података, величине речника и начина токенизације. Такође је уочено да оптимизација хиперпараметара и правилна селекција архитектуре имају велики утицај на перформансе модела.

Као предлог за будући рад, могуће је унапредити систем увођењем механизма пажње (*Attention Mechanisms*), који омогућавају да модел фокусира пажњу на релевантне делове слике током генерисања текста. Поред тога, употреба већих и разноврснијих скупова података, као и прелазак на трансформер архитектуре (*Transformers*), могла би значајно побољшати квалитет генерисаних описа.

Закључно, спроведено истраживање показује да је интеграција техника рачунарског вида и обраде природног језика изводљива и ефикасна, чак и у оквиру једноставног прототипа. Реализовани модел представља основу за даље унапређење и развој напреднијих мултимодалних система који могу имати широку примену у области дубоког учења и интелигентне анализе мултимедијалних садржаја.

ЛИТЕРАТУРА

- [1] Oriol Vinyals, Alexander Toshev, Samy Bengio, Dumitru Erhan. *Show and tell: A neural image caption generator*. 20. април 2015.
(<https://arxiv.org/pdf/1411.4555>)
- [2] Pierre Sermanet, David Eigen, Xiang Zhang, Michael Mathieu, Rob Fergus, Yann LeCun. *OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks*. 24. фебруар 2014.
(<https://arxiv.org/pdf/1312.6229>)
- [3] Bill Kromydas. *Convolutional Neural Network (CNN): A Complete Guide*. 18. јануар 2023.
(<https://learnopencv.com/understanding-convolutional-neural-networks-cnn/>)
- [4] Karen Simonyan, Andrew Zisserman. *Very deep convolutional networks for large-scale image recognition*. 10. април 2015.
(<https://arxiv.org/pdf/1409.1556>)
- [5] Sepp Hochreiter, Jürgen Schmidhuber. *Long Short-Term Memory*. новембар 1997.
(https://www.researchgate.net/publication/13853244_Long_Short-Term_Memory)
- [6] Saeed Mohsen, Ahmed Elkaseer, Steffen G. Scholz. *Industry 4.0-Oriented Deep Learning Models for Human Activity Recognition*. 8. октобар 2021.
(<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9605677>)
- [7] Kishore Papineni, Salim Roukos, Todd Ward, Wei-Jing Zhu. *Bleu: a Method for Automatic Evaluation of Machine Translation*. јул 2002.
(<https://aclanthology.org/P02-1040.pdf>)